MODPKICA Java 复刻版 v1.28

开发助手 · 源码分析

2026-09-28

基于 Java + BouncyCastle 1.85 离线构建

作者：@魔帝本尊（QQ：2829969554）

目录

一、项目概述　二、源码结构总览　三、运行流程解析　四、关键常量与数据格式
五、v1.28 今日更新　六、构建与测试

• MODPKICA Java 复刻版：用 Java + BouncyCastle 1.85 离线复刻 Go 版 MODPKICA 全套 PKI 功能。
• 覆盖能力：X.509 证书签发（RSA/ECC/SM2/DSA/Ed25519/PQC-ML-DSA/RSA-PSS）、CRL 吊销列表、OCSP 在线证书状态协议、RFC3161 与微软 Authenticode 时间戳、RFC 6962 CT 日志服务（SCT 服务管理）、WebAdmin 浏览器管理面板（33 个 API）、Windows 代码签名（Authenticode）、PDF 电子签章（Adobe / 国密 35275 / 38540 / 0031-2025）、证书查看器（仿 Windows）。
• 技术栈：纯 javac 编译，无 Maven / Gradle；依赖仅 BC185\（BC 1.85 全家桶：bcprov/bcpkix/bcutil/bcmail）+ lib\（pdfbox-app-3.0.4.jar），离线可开发。
• 平台：Windows（build.cmd）/ Linux（build.sh）；管理端口写 PKI/CONFIG.txt 的 SUPORT（默认 8080）。
• 版本：v1.28（2026-09-28）。作者：@魔帝本尊（QQ：2829969554）。
• 铁律：严禁执行 ResetData.cmd / ResetData.sh / ResetExe 等重置脚本；SHA1 保持可用仅警告、前端禁止 alert；存储方式不改、不加密、无 API 鉴权、仅监听 127.0.0.1；证书过期不影响信任关系（离线百年验证）。

二、源码结构总览

• 源码位于 src\modpkica\，共 23 个类；编译产物 out\；单测 debug_test\ModpkicaUnitTest.java。

主程序（7 个，对应 Go 版 7 个 exe）

• MAIN（853 行）： 网络服务主程序--HTTP 服务（OCSP / CRL / 时间戳 / CT / 静态网页）+ 内嵌 WebAdmin。
• Admin（431 行）： 管理控制台（init / makeroot / signcert / revoke / signCRL / initOCSP / initTIMESTAMP / verifyCERT / list）。
• MakeRoot（569 行）： 签发根证书（类型 R）。
• MakeCert（1184 行）： 签发下级证书（中间 CA 类型 C / 终端证书类型 E），支持 SCT 预证书流程。
• RootGetCrl（334 行）： 遍历 CERTS.txt 中全部 R/C 证书，为每个颁发者生成 CRL。
• CertVerify（1848 行）： 证书验证--全字段解析、签名验证、证书链、CRL+OCSP 吊销状态、SCT 逐条验签、哈希计算。
• Auto（235 行）： 自动化命令（init / revoke / restore / newcert）。

支撑工具类

• WebAdmin（7710 行）： 浏览器管理面板（前端内嵌文本块 + HttpServer 路由 + 33 个 API）。
• Bc（26 行）： 注册 BC Provider（Security.addProvider）。
• AppPaths（51 行）： 工作目录定位（系统属性/环境变量 MODPKICA_BASE -> user.dir）。
• AppEnv（46 行）： 可执行程序清单（7 个主程序名）。
• Config（181 行）： CONFIG.txt 配置加载（WEBPORT/OCSP/CRL/CRT/CPS/CPS_Nonice/SUPORT）。
• CertsDB（324 行）： CERTS.txt 登记表加载/查询/吊销/原子写回（跳过 # 注释行，带文件锁）。
• MLog（103 行）： 追加式操作日志（时间/时间戳/操作/命令/结果，双换行分隔）。
• PemIO（318 行）： PEM 证书/私钥读写、密钥类型探测、PQC OID 判断。
• CertUtil（634 行）： 密钥对生成（RSA/ECC/SM2/DSA/Ed25519/ML-DSA）、签名算法 OID、SKI 计算、ContentSigner 工厂。
• CrlUtil（132 行）： CRL 生成与 DER->PEM 转换（含吊销证书项封装）。
• OcspUtil（374 行）： OCSP 请求解析 / 无签名响应（unauthorized）。
• TimestampUtil（264 行）： RFC3161 请求解析、SM3 摘要计算器、TSA 策略 OID。
• Sm2Util（242 行）： 国密 SM2 签名/验签/SM3/ZA 计算、密钥转换。
• CtLog（974 行）： CT 日志密钥库 CRUD（ECDSA/RSA/SM2）、RFC 6962 预证书 SCT 签名、SCTList 编码、权威 log_list.json 拉取、类型匹配选组（<=3 组）。
• SignTool（2808 行）： EXE/PDF 签名与验证（Authenticode 手工 ASN.1、CAdES-T 时间戳、Adobe/国密 PDF 签章、SES_Signature 验证、LTV/DSS）。
• X509JsonConverter（2253 行）： X.509 证书 互转 JSON 结构化文本（字节级可逆往返，99.99% 全算法覆盖）。

三、运行流程解析

MAIN 启动流程

• main -> AppPaths.baseDir() 定位 PKI 根 -> 读 CONFIG.txt（WEBPORT/SUPORT）-> 注册 BC（Bc.init）-> 启动时刷新全部 CRL -> 启动 HTTP 服务（WEBPORT 端口）-> 注册路由 -> 启动内嵌 WebAdmin（SUPORT 端口，仅绑定 127.0.0.1）。
• 路由：/OCSP、/ocsp（POST，OCSP 响应）-> /CRL/{CID}.crl、/CRL/{CID}.der.crl（CRL 下载）-> /CRT/{CID}.crt（证书下载）-> /rfc3161（RFC3161 时间戳，SHA1/SHA256/SM3 自适应）-> /timestamp（Authenticode SHA256）、/timestamp/sha1（Authenticode SHA1）-> /ct/add（RFC 6962，仅 POST）-> /（静态网页 PKI/WebPublic）。
• 每次请求前重新加载 CERTS.txt（外部修改即时生效）。

OCSP 请求处理流程

• 解析请求（OcspUtil.parseRequest）-> 提取哈希算法 OID 与证书序列号 -> CertsDB 查登记（跳过 # 行）-> 未知序列号返回无签名 unauthorized 响应 -> 已知则按颁发者序列号加载 OCSP/序列号_ocsp.crt|.key 密钥对 -> 构造 CertID 与响应（RSA/ECC 按请求摘要算法 SHA1/256/384/512，缺省 SHA1；SM2 固定 SM3）-> SM2 分支按 RFC 6960 标准编码 revoked [1] IMPLICIT RevokedInfo -> 签名返回。

时间戳请求处理流程

• RFC3161：解析 TimeStampReq（TimestampUtil.parseRfc3161Request）-> 校验 messageImprint 算法（SHA1/SHA256/SHA384/SHA512/SM3）-> 按摘要算法从 TIMSTAMP\ 加载自适应 TSA 证书（sha1->rsa_sha1/ecc_sha1 随机；sha256->rsa_sha256/ecc_sha256 随机；SM3->sm2_sm3）-> 生成 TimeStampToken（含 Nonce 回显、TSA 证书、签名）-> 返回 TimeStampResp。
• Authenticode：请求体 base64（容忍 CRLF/末尾 NUL）-> 微软 PKCS#7 分离签名（countersignature 机制）-> 提取 token 首个 SignerInfo 嵌入 1.2.840.113549.1.9.6，并并入 TSA 证书。

证书签发流程（MakeCert）

• 解析参数（DN / 类型 / 密钥算法/位数 / 哈希 / EKU / KeyUsage / 证书策略 / pathLen / SAN / 有效期）-> 按 X.500/RFC 规范顺序排序 DN -> 生成密钥对 -> 按颁发者序列号从 ROOT/CA 目录加载颁发者证书+私钥 -> 组装扩展（SKI/AKI/AIA/CRL 分发点/证书策略/EKU/SCT 等）-> 若为服务器/客户端认证证书：先签发含 poison 的 SCT 预证书 -> POST 本机 /ct/add -> 取回 SCTList 嵌入 SCT 扩展 -> 二次签名生成最终证书 -> 按类型写 ROOT/CA/CERT/KEY 目录并同步 WebPublic/CRT。

证书验证流程（CertVerify / WebAdmin /api/verify）

• 解析证书（文件或序列号）-> 打印全字段 -> 验证签名（颁发者公钥）-> 证书链（R->C->E，含 pathLenConstraint 中文提示）-> CRL 吊销状态（读 WebPublic/CRL）-> OCSP 吊销状态（请求 CONFIG OCSP 地址）-> SCT 列表逐条验签（按 log_id 匹配本地 CTLOG 密钥库 + 权威 log_list.json，precert_entry/x509_entry 待签名数据重算）-> 输出各类哈希（SHA1/SHA256、SKI 三种、Pin-SHA256 等）。

四、关键常量与数据格式

PKI 目录与命名规则

- PKI/ROOT\：根证书 序列号.crt + 序列号.key（类型 R）。
- PKI/CA\：中间证书 序列号.crt + 序列号.key（类型 C）。
- PKI/CERT\：终端证书 PEM 序列号.crt；PKI/KEY\：终端私钥 序列号.key（类型 E）。
- PKI/OCSP\：序列号_ocsp.crt + 序列号_ocsp.key。
- PKI/TIMSTAMP\：类型_哈希算法.crt|.key（rsa_sha1/rsa_sha256/ecc_sha1/ecc_sha256/sm2_sm3）。
- PKI/WebPublic/CRT\：全部证书对外下载 序列号.crt；PKI/WebPublic/CRL\：全部 CRL（.crl/.der.crl）。
- PKI/CTLOG\：modpkica_ct_list.json（CT 日志密钥库，v3 风格，含 type 字段）+ ctlogs.txt（请求日志，只追加）。
- PKI/CERTS.txt、CONFIG.txt、logs.log、MY\（我的证书导入库 myCERTS.txt）。

CERTS.txt 行格式

- 序列号 类型(R/C/E) 状态(V/R/N) 原因编号(0-9) 时间(yyyy/MM/dd-HH:mm:ss) 颁发者序列号 算法。
- 行首 # 为注释/忽略（CertsDB.load 跳过；WebAdmin 右键「删除证书」即加 # 前缀）。
- 示例：13fbe86586d1bca R V 0 2026/09/08-23:18:34 13fbe86586d1bca RSA

CONFIG.txt 键

- WEBPORT（WEB 服务端口，默认 80）；SUPORT（管理面板端口，默认 8080）。
- OCSP / CRL / CRT：AIA / CRL 分发点 URL，{CID} 占位符替换为序列号。
- CPS：证书策略 CPS URI；CPS_Nonice：用户通告文本（多行用 \n 存储，签发时还原换行）。

核心 OID 常量

- 签名算法：SHA256-RSA 1.2.840.113549.1.1.11；RSA-PSS 1.2.840.113549.1.1.10；ECDSA-SHA256 1.2.840.10045.4.3.2；SM3-SM2 1.2.156.10197.1.501（Sm2Util.OID_SM3_WITH_SM2）；SM3 1.2.156.10197.1.401（SignTool.OID_SM3）；SM2 曲线 1.2.156.10197.1.301；ML-DSA-44/65/87 2.16.840.1.101.3.4.3.17/18/19。
- 扩展：SCT 2.25.29.14（SKI）/ 2.5.29.35（AKI）/ 2.5.29.17（SAN）/ 2.5.29.31（CRL 分发点）/ 2.5.29.32（证书策略）/ 2.5.29.37（EKU）；SCT 列表 1.3.6.1.4.1.11129.2.4.2（CtLog.OID_SCT）；poison 1.3.6.1.4.1.11129.2.4.3（CtLog.OID_POISON）；OCSP Nonce 1.3.6.1.5.5.7.48.1.2（OcspUtil.OID_NONCE）；TSA 时间戳属性 1.2.840.113549.1.9.16.2.14（SignTool.OID_TSA_TOKEN）；Authenticode SPC 1.3.6.1.4.1.311.2.1.4 / 1.3.6.1.4.1.311.2.1.15 / 1.3.6.1.4.1.311.3.3.1；国密政务扩展 1.2.156.112600.8.1（SignTool.OID_GOV_EXT）。
- 证书策略：基础 CPS 1.3.6.1.4.1.4146.1.95；SSL EV 2.23.140.1.1 / DV 2.23.140.1.2.1 / OV 2.23.140.1.2.2 / IV 2.23.140.1.2.3；代码签名标准 2.23.140.1.4.1 / EV 2.23.140.1.3；TSA 2.23.140.1.4.2；文档签名通用 1.3.6.1.5.5.7.3.36 / 微软 1.3.6.1.4.1.311.10.3.12。
- EKU：服务器/客户端认证 1.3.6.1.5.5.7.3.1/2；代码签名 1.3.6.1.5.5.7.3.3（SignTool.EKU_CODE_SIGNING）；时间戳 1.3.6.1.5.5.7.3.8；OCSP 签名 1.3.6.1.5.5.7.3.9；微软文档签名 1.3.6.1.4.1.311.10.3.12（EKU_DOC_MICROSOFT）；通用文档签名 1.3.6.1.5.5.7.3.36（EKU_DOC_GENERIC）；内核/驱动/系统组件/OEM/内嵌 1.3.6.1.4.1.311.61.1.1 / 1.3.6.1.4.1.311.10.3.5~8。

SCT 枚举（CtLog）

- ECDSA：hash=4(SHA256) sig=3(ECDSA)；RSA：hash=4 sig=1；SM2：hash=7(SM3) sig=4（国密 CT 生产生态，对齐零信 sm2ct.cn；兼容 sig=7）。
- SCT 待签名数据 = version + signature_type(0) + timestamp(8B) + entry_type(2B) + signed_entry + extensions(2B+数据)；precert_entry 含 issuer_key_hash=SHA-256(颁发者 SPKI DER) + 去 poison TBS。
- 选组规则：SM2 预证书只选 usable 的 SM2 密钥；RSA/ECDSA 预证书只选 RSA+ECDSA 密钥；随机 <=3 组（仅一组就签一组）。

签名专区 SubFilter

• Adobe 标准：adbe.pkcs7.detached；国密 35275：GM.sm2cms.detached；国密 38540 / 0031-2025：GM.sm2seal；纯文档时间戳：ETSI.RFC3161。

五、v1.28 今日更新（2026-09-28）

• 基于 20260927 逐文件逐函数源码审计报告：完成 P1-01/P1-03/P1-05（证书验证与解析时间无关性--证书过期不影响信任关系，离线百年验证）+ P2 系列（单文件上传上限 20MB、总请求体 256MB、JSON 4MB；-Xlint:all 警告 396 条清零；build 1-9 编号）。
• 签发证书页：中间 CA 新增「路径长度」可编辑下拉（默认 null/0-9，MakeCert a[12]=pathlen -> BasicConstraints pathLen）；终端证书新增「证书策略」8 复选框（SSL DV/IV/OV/EV、代码签名标准/EV、文档签名、TSA），仅勾选写入，基础 CPS 策略保留；Admin.doSignCert 14 参重载 + 12 参 CLI 兼容。
• CertVerify 链验证提示中文化（pathLenConstraint=0 -> 「因为证书路径中有一个证书颁发机构似乎没有颁发证书的权限或不允许用作终端实体证书,所以此证书无效」）。
• 证书库 CERTS.txt 右键「删除证书」：二次确认 -> 不删文件，CERTS.txt 该行加 # 注释（忽略），生成 CRL/OCSP 时自动跳过（CertsDB 统一跳过 # 行）；新增 API /api/certs/delete（共 33 个 API）。
• 验证：-Xlint:all 零警告；冒烟 TestCertsDelete.java 8 项 PASS；单测 131 passed, 0 failed。
• Web 面板左下角版本号：MODPKICA Java 复刻版 v1.28（Admin.VERSION）。

六、构建与测试

- 编译：javac -Xlint:all -encoding UTF-8 -cp "out;BC185*;lib*" -d out src\modpkica*.java（零警告）。
- 一键脚本：build.cmd / build.sh，支持编号 1-9，编译成功后自动运行对应命令，编译失败不运行。
- 编号：1 MAIN、2 WebAdmin、3 Admin、4 MakeRoot（示例参数）、5 MakeCert（示例参数）、6 RootGetCrl、7 CertVerify cert.crt、8 Auto revoke 示例、9 ModpkicaUnitTest（单测）。
- 单元测试：debug_test\ModpkicaUnitTest.java，131 passed / 0 failed（覆盖 X509JsonConverter 全算法往返、PSS/SHA3、SCT 验签、OCSP 等）。
- 冒烟测试：debug_test\TestCertsDelete.java、TestSignPolicy.java 保留于库。
- 运行统一入口：java "-Dfile.encoding=UTF-8" -cp "out;BC185*;lib*" modpkica.主程序名 [参数]。
- 管理端口：默认 8080；CONFIG.txt SUPORT 可改；-Dmodpkica.admin.port=8082 显式覆盖。

七、逐类 API 速查（常量 · 字段 · 方法 · 运行例子）

7.1 MAIN（网络服务主程序）

• 类声明：public final class MAIN
• 常量：无公开常量；端口与地址全部来自 CONFIG.txt / 系统属性。
• 方法 main(String[] args)：启动全部服务。
• 运行例子：java "-Dfile.encoding=UTF-8" -cp "out;BC185*;lib*" modpkica.MAIN
• 说明：启动后输出各服务地址横幅；管理面板 http://localhost:8080；OCSP/CRL/时间戳/CT 路由见第三章。
• 内部主要私有方法（仅概述）：handleOcsp（OCSP 处理，按 CERTS.txt 加载 序列号_ocsp 密钥）、handleCrl / handleCrt（文件下载）、handleRfc3161（RFC3161 时间戳，自适应 TSA 证书）、handleAuthenticode（微软时间戳，SHA1/SHA256）、handleCtAdd（RFC 6962 SCT 签发）、refreshAllCrl（启动时刷新 CRL）、startWebAdmin（内嵌面板，端口 = CONFIG SUPORT）。

7.2 Admin（管理控制台）

• 类声明：public final class Admin
• 常量：
• VERSION = "MODPKICA Java 复刻版 v1.28 (BC 1.85 离线)"：版本信息（Web 面板左下角读取 /api/status）。
• 方法（全部 static）：
• main(String[] args)：交互式命令行入口。
• doInit(baseDir, dn, bit, hash, algo)：初始化环境（清空证书目录 + 重建 CERTS.txt 模板，备份旧台账）。
• doMakeRoot(baseDir, dn, bit, hash, algo)：签发根证书（参数：证书 DN、密钥位数、哈希算法、密钥算法）。
• doMakeRoot(baseDir, dn, bit, hash, algo, evoid)：签发根证书（额外 evoid = EV 组织标识符 OID）。
• doList(baseDir)：列出全部登记证书。
• doRevoke(baseDir, serial, reason)：吊销证书（reason 0-9）。
• doRestore(baseDir, serial)：恢复证书（状态 R->V、原因置 0，同步生成 CRL）。
• doSignCrl(baseDir)：刷新全部 CRL。
• doInitOcsp(baseDir, algo)：为全部 R/C 证书签发 OCSP 签名证书（写 OCSP/序列号_ocsp.crt|.key）。
• doInitTimestamp(baseDir, hash)：签发时间戳证书全家桶（rsa/ecc/sm2 × sha1/sha256/sm3）。
• doVerifyCert(baseDir, target)：证书验证（文件或序列号）。
• 运行例子：java "-Dfile.encoding=UTF-8" -cp "out;BC185*;lib*" modpkica.Admin
• 交互命令示例：makeroot "CN=Root,O=My Org,C=CN" 2048 sha256 RSA / signcert "CN=www.test.com" 2048 sha256 1 1 0 1 "www.test.com" null null root RSA / revoke 113521bfd 5 / signCRL / initOCSP RSA / initTIMESTAMP sha256 / verifyCERT 113521bfd

7.3 MakeRoot（签发根证书）

• 类声明：public final class MakeRoot
• 方法（static）：
• main(String[] args)：CLI 入口（参数：DN 密钥位数 哈希算法 密钥算法）。
• run(baseDir, args)：按参数执行签发：生成密钥对（RSA/ECC/SM2/DSA/EdDSA/PQC/RSASSA-PSS 分支）-> DN 规范排序 -> 组装扩展（SKI/AKI 自指、AIA、CRL 分发点、证书策略 CPS+UserNotice、基本约束 CA:true）-> 自签 -> 写 ROOT/序列号.crt|.key + WebPublic/CRT/序列号.crt -> 登记 CERTS.txt（类型 R）。
• 运行例子：java "-Dfile.encoding=UTF-8" -cp "out;BC185*;lib*" modpkica.MakeRoot "CN=Root,O=My Org,C=CN" 2048 sha256 RSA
• 说明：密钥算法可选 RSA / ECC / SM2 / DSA / EDDSA / PQC / RSAPSS；哈希算法对应可选 SHA1/SHA256/SHA384/SHA512/SM3/NULL；PQC 位数选 ML-DSA-44/65/87。

7.4 MakeCert（签发下级证书）

• 类声明：public final class MakeCert
• 方法（static）：
• setQuiet(boolean q)：服务安静模式开关（WebAdmin 操作期间置位，生成信息只写日志不打印控制台）。
• main(String[] args)：CLI 入口（12 参兼容格式）。
• run(baseDir, a)：核心签发。参数布局 a[0]=DN、a[1]=密钥位数、a[2]=哈希、a[3]=证书类型(1=CA,0=终端,2=根,3=OCSP,4=TSA)、a[4]=有效期年数、a[5]=SAN 类型、a[6]=SAN 内容、a[7]=颁发者序列号(root 或 序列号)、a[8]=密钥算法、a[9]=EKU、a[10]=KeyUsage、a[11]=证书策略 OID 列表、a[12]=pathLen（中间 CA 路径长度，null 不限制）、a[13]=EV 组织标识符 OID。
• 流程：按类型加载颁发者（ROOT/CA 目录）-> 生成密钥 -> 组装扩展 -> 若为服务器/客户端认证证书则走 SCT 预证书流程（CtLog）-> 二次签名 -> 按类型写 ROOT/CA/CERT/KEY/OCSP/TIMSTAMP + WebPublic/CRT -> 登记 CERTS.txt。
• 运行例子：java "-Dfile.encoding=UTF-8" -cp "out;BC185*;lib*" modpkica.MakeCert "CN=www.test.com" 2048 sha256 1 1 0 1 "www.test.com" null null root RSA
• 说明：证书类型 1=中间 CA（写 CA/）、0=终端（写 CERT/+KEY/）；OCSP 证书写 OCSP/序列号_ocsp；TSA 证书写 TIMSTAMP/类型_哈希.crt。

7.5 RootGetCrl（签发吊销列表）

• 类声明：public final class RootGetCrl
• 方法（static）：
• main(String[] args)：CLI 入口。
• run()：以当前工作目录为 PKI 根运行。
• run(baseDir)：遍历 CertsDB 全部 R/C 证书，为每个颁发者生成独立 CRL（WebPublic/CRL/序列号.crl + .der.crl），哈希算法沿用颁发者证书；CRL 含吊销证书列表（序列号/时间/原因）。
• 运行例子：java "-Dfile.encoding=UTF-8" -cp "out;BC185*;lib*" modpkica.RootGetCrl
• 说明：# 注释行证书自动跳过（CertsDB.load）。

7.6 CertVerify（证书验证）

• 类声明：public final class CertVerify
• 方法（static）：
• main(String[] args)：CLI 入口（参数 = 证书文件或序列号）。
• run(filename)：完整验证--解析全字段、验签、证书链（含 pathLen 中文提示、信任锚 PKI/ROOT + JAVA cacerts）、CRL/OCSP 吊销状态、SCT 列表逐条验签（本地 CTLOG + 权威 log_list.json）、输出 SHA1/SHA256/SKI/Pin-SHA256 哈希。
• 运行例子：java "-Dfile.encoding=UTF-8" -cp "out;BC185*;lib*" modpkica.CertVerify cert.crt / modpkica.CertVerify 113521bfd
• 说明：内部私有方法要点：chainConstraintsOk（pathLenConstraint 检查，中文提示）、verifySctSignature（SCT 验签，ECDSA DER+裸 r||s / RSA / SM2 (7,4)(7,7)）、buildSerializedSct（待签名数据）、sctStatus（状态判定）、inJavaCacerts（cacerts 信任判定）。

7.7 Auto（自动化命令）

• 类声明：public final class Auto
• 方法（static）：
• main(String[] args)：CLI 入口（子命令 init / revoke / restore / newcert）。
• runInit(baseDir)：初始化环境（备份 CERTS.txt -> 写模板 + myCERTS.txt.sm 模板）。
• revoke(baseDir, serial, reason)：吊销（改写 CERTS.txt + 刷新 CRL）。
• restore(baseDir, serial)：恢复（R->V、原因 0，同步 CRL）。
• 运行例子：java … modpkica.Auto init / modpkica.Auto revoke 113521bfd 5 / modpkica.Auto newcert 序列号 E V 0 null 13fbe86586d1bca RSA

7.8 WebAdmin（浏览器管理面板）

• 类声明：public final class WebAdmin
• 方法（static）：
• main(String[] args)：独立启动面板（探测 MAIN 状态，读 CONFIG SUPORT/WEBPORT）。
• port()：当前管理端口。
• start(String mainWebPort)：启动面板（MAIN 内嵌自动调用；端口优先级 -Dmodpkica.admin.port -> CONFIG SUPORT -> 8080；仅绑定 127.0.0.1）。
• 说明：前端 = 内嵌 text block（PAGE_P1/P2...，深色主题，15 页面）；后端 = HttpServer + 8 线程执行器 + WRITE_LOCK 写串行锁；33 个 API（/api/status /api/certs /api/certs/delete /api/certs/cacerts /api/logs /api/config /api/chain /api/certjson /api/version /api/issuers /api/init /api/makeroot /api/signcert /api/revoke /api/restore /api/rekey /api/renew /api/crl /api/initocsp /api/inittimestamp /api/verify /api/signcerts /api/sign/exe|pdf /api/verify/exe|pdf /api/debug/ocsp|tsa /api/ctlog/list|logs|gen|delete|enable|pause /api/my/import|delete）。
• 关键私有方法：certsJson（CERTS.txt 列表）、cacertsJson（JAVA cacerts 列表）、myDelete（我的证书删除）、certsDelete（CERTS.txt 注释式删除，v1.28 新增）、statusJson（状态+版本 Admin.VERSION）、configJson/saveConfig、signAction（EXE/PDF 签名分发）、verifyAction、ctLog 系列、ensureSuportConfig（CONFIG 自动建 SUPORT）。
• 运行例子：java "-Dfile.encoding=UTF-8" -cp "out;BC185*;lib*" modpkica.WebAdmin（访问 http://localhost:8080）

7.9 X509JsonConverter（证书 互转 JSON 结构化）

• 类声明：public class X509JsonConverter
• 方法（static）：
• main(String[] args)：CLI 转换入口。
• certToJson(byte[] pemBytes)：证书 PEM -> JSON 结构化文本（version/serial/signature/issuer/validity/subject/publicKey/extensions/SCT 等全字段）。
• jsonToCertPem(String json)：JSON -> PEM 证书（字节级可逆；先做往返检测，通过则尽量省略 raw 减小体积，不通过的部分保留原始 DER raw）。
• generateKeyPairFromJson(String json)：从 JSON 公钥参数恢复密钥对。
• keyTypeFromJson(String json)：JSON -> 密钥类型（RSA/ECC/SM2/PQC...）。
• 说明：支持现有所有算法证书（RSA/ECC/SM2/DSA/Ed25519/ML-DSA/RSASSA-PSS），市面 99.99% 证书 Asn.1/x509 结构化可逆序列化。
• 运行例子：java "-Dfile.encoding=UTF-8" -cp "out;BC185*;lib*" modpkica.X509JsonConverter cert.crt
• 调用示意：String json = X509JsonConverter.certToJson(pemBytes); byte[] pem = X509JsonConverter.jsonToCertPem(json);

7.10 CertsDB（证书登记表）

• 类声明：public class CertsDB
• 字段：entries（内存条目列表）、dbPath、lastLoadWarning；条目 CertEntry：serial/type(R/C/E)/status(V/R/N)/reason/time/issuerId/algo（含 keyType/hash）。
• 方法：
• load(String baseDir)：加载 CERTS.txt（跳过空行与 # 注释行，非法行计数跳过）。
• reload()：重新加载（外部修改即时生效）。
• findBySerial(serial)：按序列号查条目。
• findByIssuerAndSerial(issuerId, serial)：按颁发者+序列号查。
• findIssuerIdBySerial(serial)：反查颁发者序列号。
• addEntry(e)：追加条目（锁内写回文件末尾）。
• revokeCert(serial, reason, time)：吊销（内存+文件同步）。
• getAllRootAndCA()：全部 R/C 条目。
• getAllRevoked()：全部 R 状态条目。
• getAll()：全部条目（只读语义）。
• 说明：写回走 rewriteAll（tmp -> fsync -> .bak -> 原子替换，CERTS.txt.lock 跨进程锁）；v1.28 注释行机制依赖 load 跳过 #。
• 调用示意：CertsDB db = new CertsDB(); db.load(baseDir); CertsDB.CertEntry e = db.findBySerial("113521bfd");

7.11 Config（CONFIG.txt 配置）

- 类声明：public class Config
- 字段：raw（键值 Map）。
- 方法：
- Config(baseDir)：构造并加载。
- load(baseDir)：读取 CONFIG.txt（无文件用默认模板，SUPORT 缺失自动补 8080）。
- getCpsWithCid(certId)：CPS URI 替换 {CID}。
- getCrlUrl() / getCrtUrl() / getOcspUrl()：CRL/CRT/OCSP 模板 URL。
- getCps() / getCpsNonice()：证书策略 CPS 与用户通告（\n 还原换行由签发侧处理）。
- getWebPort()：WEB 端口（默认 80）。
- getRaw()：原始键值 Map。
- 调用示意：Config cfg = new Config(baseDir); String ocsp = cfg.getOcspUrl();

7.12 MLog（操作日志）

- 类声明：public final class MLog
- 方法（static）：
- append(baseDir, operation, result)：写一条日志（自动附加时间 + 时间戳）。
- append(baseDir, operation, command, result)：写日志（含输入命令）。
- appendErr(baseDir, operation, command, error)：写错误日志。
- 说明：logs.log 只追加；每条含 时间/时间戳/操作/命令/结果，记录间两个换行分隔；线程安全（synchronized）。
- 调用示意：MLog.append(baseDir, "WebAdmin /api/certs/delete", "POST 删除证书 serial=xxx", "行已注释");

7.13 Bc（BC Provider 注册）

- 类声明：public final class Bc
- 方法：init()：Security.addProvider(new BouncyCastleProvider())（幂等）。
- 调用示意：Bc.init();

7.14 AppPaths（工作目录定位）

- 类声明：public final class AppPaths
- 方法：baseDir()：返回 PKI 根--优先级：系统属性 MODPKICA_BASE -> 环境变量 MODPKICA_BASE -> user.dir。
- 调用示意：String baseDir = AppPaths.baseDir();

7.15 AppEnv（可执行程序清单）

- 类声明：public final class AppEnv
- 方法：getExeNames()：返回 7 个主程序类名数组（MAIN/Admin/MakeRoot/MakeCert/RootGetCrl/CertVerify/Auto）。

7.16 PemIO（PEM 读写与密钥探测）

- 类声明：public final class PemIO
- 方法（static）：
- readCert(path)：读 PEM/DER 证书 -> X509Certificate。
- readPrivateKey(path)：读 PEM 私钥 -> AsymmetricKeyParameter（BC 底层参数）。
- readPrivateKeyAsSigner(path)：读 PEM 私钥 -> ContentSigner（可直接用于签发）。
- writeCert(path, cert)：写 PEM 证书。
- writePrivateKey(path, key, keyType)：写 PEM 私钥（按 keyType 选编码，支持 RSA/EC/SM2/DSA/Ed25519/ML-DSA 等）。

• detectKeyType(pemPath)：探测私钥类型（读 PEM 头/PKCS8 结构）。
• isPqcOid(oid)：判断是否 PQC（ML-DSA）算法 OID。
• isPqcAlgName(alg)：判断是否 PQC 算法名。
• 调用示意：X509Certificate c = PemIO.readCert("PKI/ROOT/6cfbd3ad.crt");

7.17 CertUtil（密钥与证书构建工具）

• 类声明：public final class CertUtil
• 方法（static）：
• generateRsaKeyPair(int bits)：生成 RSA 密钥对（1024~8192）。
• generateEccKeyPair(int bits)：生成 EC 密钥对（P-256/384/521，bits 决定曲线）。
• generateSm2KeyPair()：生成 SM2 密钥对（sm2p256v1）。
• generateDsaKeyPair(int bits)：生成 DSA 密钥对。
• generateEd25519KeyPair()：生成 Ed25519 密钥对。
• generatePqcKeyPair(spec)：生成 ML-DSA 密钥对（ML-DSA-44/65/87，BC 1.85 MLDSAParameterSpec.fromName）。
• normalizePqcSpec(spec)：规格名归一化（大小写/连字符容错）。
• getSignatureAlgOid(keyType, hash)：按密钥类型+哈希返回签名算法 OID（RSA-PSS/SHA256-RSA/ECDSA/SM3-SM2/EdDSA/ML-DSA/NULL 等）。
• readJcaPrivateKey(path)：读 JCA 私钥。
• createContentSigner(key, keyType, hash)：创建 ContentSigner（BC 签发核心；SM2 走内部 SM2ContentSigner，Ed25519/ML-DSA 走对应签名器）。
• getSubjectKeyIdentifier(pubKey)：计算 SKI（RFC 5280 方法 1：SHA-1 公钥位串）。
• 内部类 SM2ContentSigner：getAlgorithmIdentifier/getOutputStream/getSignature--SM2 签名器（ZA+SM3，GB/T 32918）。
• 调用示意：KeyPair kp = CertUtil.generateEccKeyPair(256); ContentSigner cs = CertUtil.createContentSigner(kp.getPrivate(), "ECC", "sha256");

7.18 CrlUtil（CRL 工具）

• 类声明：public final class CrlUtil
• 方法（static）：
• derToPem(byte[] der)：CRL DER -> PEM。
• 内部类 RevokedCert：构造 RevokedCert(BigInteger serial, Date revocationTime, int reason)--吊销证书项封装。
• 调用示意：byte[] pem = CrlUtil.derToPem(crlDer);

7.19 OcspUtil（OCSP 工具）

• 类声明：public final class OcspUtil
• 常量：
• OID_NONCE = "1.3.6.1.5.5.7.48.1.2"：Nonce 扩展 OID。
• OID_BASIC：BasicOCSPResponse OID（1.3.6.1.5.5.7.48.1.1）。
• GOOD = 0 / REVOKED = 1 / UNKNOWN = 2：证书状态常量。
• 方法（static）：
• parseRequest(byte[] der)：解析 OCSP 请求 -> OcspRequest（序列号、颁发者哈希、Nonce、哈希算法 OID）。
• unauthorizedResponse()：无签名响应（未知序列号时返回，RFC 6960）。
• 说明：响应生成在 MAIN 内（OcspUtil + BC OCSPServer）；SM2 分支手动 ASN.1 编码 revoked [1] IMPLICIT RevokedInfo。
• 调用示意：OcspUtil.OcspRequest req = OcspUtil.parseRequest(bodyBytes);

7.20 TimestampUtil（时间戳工具）

• 类声明：public final class TimestampUtil

• 常量：TSA_POLICY_OID：TSA 策略 OID。
• 方法（static）：
• parseRfc3161Request(byte[] der)：解析 RFC3161 请求 -> Rfc3161Request（messageImprint 摘要算法 + 摘要值、Nonce、certReq）。
• 内部类 SM3DigestCalculator：getAlgorithmIdentifier/getOutputStream/getDigest--SM3 摘要计算器（BC DigestCalculator 接口，供 SM2 时间戳消息摘要用）。
• 调用示意：TimestampUtil.Rfc3161Request req = TimestampUtil.parseRfc3161Request(body);

7.21 Sm2Util（国密 SM2/SM3）

• 类声明：public final class Sm2Util
• 常量：
• OID_SM3_WITH_SM2 = "1.2.156.10197.1.501"：SM3-SM2 签名算法。
• OID_SM3 = "1.2.156.10197.1.401"：SM3 摘要算法。
• OID_SM2_PUBLIC_KEY = "1.2.156.10197.1.301"：SM2 曲线（sm2p256v1）。
• 方法（static）：
• getSm2UidBytes()：默认用户标识（1234567812345678）。
• sm3(byte[] data)：SM3 摘要。
• computeZa(ECPublicKeyParameters pub, byte[] uid)：计算 ZA（GB/T 32918.1）。
• sm2Sign(ECPrivateKeyParameters priv, byte[] tbs)：SM2 签名（SM3(ZA||M)）。
• sm2Verify(ECPublicKeyParameters pub, byte[] tbs, byte[] sig)：SM2 验签（兼容 DER 与裸 r||s）。
• generateSm2KeyPair()：BC 底层 SM2 密钥对。
• generateSm2JcaKeyPair()：JCA 密钥对。
• toEcPublicParams(PublicKey pub)：JCA 公钥 -> BC 参数。
• toEcPrivateParams(PrivateKey priv)：JCA 私钥 -> BC 参数。
• main(String[] args)：自测入口。
• 调用示意：byte[] za = Sm2Util.computeZa(pub, Sm2Util.getSm2UidBytes()); byte[] sig = Sm2Util.sm2Sign(priv, tbs);

7.22 CtLog（CT 日志服务）

• 类声明：public final class CtLog
• 常量：
• OID_POISON = "1.3.6.1.4.1.11129.2.4.3"：SCT 预证书 poison 扩展。
• OID_SCT = "1.3.6.1.4.1.11129.2.4.2"：SCT 列表扩展。
• PUBLIC_LIST_URL = "https://www.gstatic.com/ct/log_list/v3/log_list.json"：权威日志公钥数据集。
• 内部类：PubLog（权威日志条目：log_id/description/key/timestamp 等）、LogKey（本地密钥：logId(base64)/description/state(usable|disable)/type(ECDSA|RSA|SM2)/privateKey/createdAt）。
• 方法（static）：
• keysJsonPath(baseDir)：PKI/CTLOG/modpkica_ct_list.json 路径。
• logsTxtPath(baseDir)：PKI/CTLOG/ctlogs.txt 路径。
• publicListPath(baseDir)：权威 log_list.json 路径（项目根/PKI/PKI/WebPublic 三级备选）。
• loadPublicList(baseDir)：读取权威公钥列表。
• publicListTimestamp(baseDir)：数据日期。
• fetchPublicList(baseDir)：在线拉取权威列表并保存。
• loadKeys(baseDir)：加载本地密钥库。
• saveKeys(baseDir, keys)：保存密钥库。
• generateKey(baseDir, desc, type)：生成 ECDSA/RSA/SM2 密钥对（log_id = SHA-256(SPKI) base64）。
• deleteKey(baseDir, logIdB64)：删除密钥。
• setState(baseDir, logIdB64, state)：启用/暂停（usable/disable）。
• pickLogKeys(baseDir, sm2Precert)：按预证书类型选组（SM2 只选 SM2；RSA/ECDSA 选 RSA+ECDSA；随机 <=3 组，仅一组就签一组）。
• isSm2Precert(X509Certificate cert)：判断预证书是否 SM2（解析 SPKI 曲线 OID 1.2.156.10197.1.301）。

• appendRequestLog(baseDir, line)：追加请求日志（只增不删）。
• readLogs(baseDir)：读请求日志全部行。
• parsePemCerts(byte[] pemBytes)：解析 PEM 证书（多张）。
• pemCert(byte[] der)：DER -> PEM。
• tbsWithoutPoison(X509Certificate cert)：删除 poison 扩展后的 TBS DER（RFC 6962 §3.1）。
• extHex(X509Certificate cert, oid)：取扩展 HEX。
• 运行例子：由 MAIN /ct/add 路由与 WebAdmin SCT服务管理页面调用；命令行无独立入口。
• 说明：SCT 签名枚举 ECDSA=(4,3) RSA=(4,1) SM2=(7,4)（兼容 (7,7)）；SCT 待签名数据构造见第四章。

7.23 SignTool（EXE/PDF 签名与验证）

• 类声明：public final class SignTool
• 常量：
• OID_SPC_INDIRECT_DATA = "1.3.6.1.4.1.311.2.1.4"：Authenticode SpcIndirectDataContent。
• OID_SPC_PE_IMAGE_DATA = "1.3.6.1.4.1.311.2.1.15"：SpcPeImageData。
• OID_SPC_RFC3161_TS = "1.3.6.1.4.1.311.3.3.1"：微软 RFC3161 时间戳属性。
• OID_SM3 = "1.2.156.10197.1.401" / OID_SM2 = "1.2.156.10197.1.501"。
• OID_GOV_EXT = "1.2.156.112600.8.1"：国密电子印章扩展。
• OID_TSA_TOKEN = "1.2.840.113549.1.9.16.2.14"：CAdES-T 签名时间戳属性（id-aa-timeStampToken）。
• SUBFILTER_GM_SEAL = "GM.sm2seal" / SUBFILTER_GM_CMS = "GM.sm2cms.detached" / SUBFILTER_ETSI_RFC3161 = "ETSI.RFC3161"。
• EKU_CODE_SIGNING / EKU_KERNEL_MODE / EKU_DRIVER_VERIFY / EKU_SYS_COMPONENT / EKU_OEM_SYS / EKU_EMBED_SYS / EKU_DOC_MICROSOFT / EKU_DOC_GENERIC：EKU 常量（见第四章 OID 表）。
• 方法（static）：
• isCodeSignCert(cert)：证书是否代码签名用途（EKU 交集）。
• isDocSignCert(cert)：证书是否文档签名用途（微软或通用 OID）。
• ekusOf(cert)：取 EKU 集合。
• keyBits(cert)：证书公钥位数。
• loadCertBySerial(baseDir, serial)：按序列号加载证书（R->ROOT、C->CA、E->CERT、MY 兜底）。
• loadKeyBySerial(baseDir, serial)：按序列号加载私钥（E->KEY、MY 兜底）。
• loadChain(baseDir, serial)：按颁发者关系加载完整证书链。
• digestName(keyType) / digestJca(keyType) / digestOid(keyType)：摘要算法名/JCA 名/OID（RSA/ECC->SHA256，SM2->SM3，PQC->内置）。
• sigAlgOid(keyType) / jcaSigAlg(keyType)：签名算法 OID / JCA 名。
• verifyExe(byte[] exe)：验证 PE 文件 Authenticode 签名（CMS 验签 + 文件哈希比对 + 证书链 + 时间戳）。
• signPdfTimestampOnly(byte[] pdf, tsUrl)：纯文档时间戳（无证书，ETSI.RFC3161 + DSS/LTV）。
• applyLtvPdf(byte[] signedPdf, chain, baseDir)：为已签名 PDF 叠加 DSS/LTV。
• verifyPdf(byte[] pdf)：验证 PDF（逐签名字段：CMS/DocTimeStamp/SES_Signature，含 CAdES-T 时间戳解析与 LTV 状态）。
• 内部类/私有方法（概述）：signExe（PE/COFF 嵌入 WIN_CERTIFICATE + 手工 ASN.1 SpcIndirectData）、requestTimestamp（RFC3161）、requestAuthenticodeTimestamp（微软 base64 请求）、buildCms（CMS 构造，unsignedAttrs 时间戳）、buildExeCms（countersignature 分支）、signPdfGm（国密三标准）、buildGmSealInfo38540/35275、buildGm38540Signature（SES_Signature，第 5 字段 timeStamp [0] BIT STRING）、verifyGmSeal38540（判别 38540/0031）、collectLtvData/installDss（DSS 字典）。
• 运行例子：由 WebAdmin /api/sign/exe、/api/sign/pdf、/api/verify/exe、/api/verify/pdf 调用；命令行无独立入口。
• 说明：Adobe 模式选中证书时时间戳按 CAdES-T 标准嵌入签名内部（messageImprint = SHA-256(签名值)）；不选证书时生成独立 DocTimeStamp；国密 35275/38540/0031 需 SM2 证书 + 签章图片；时间戳失败自动降级不影响主签名。